

I have to format the data which is downloading from data base in to XML format This is the opcua XML format I have a template or sample XML So the field name should be same as the sample one, and since the data is coming from data base value of each field could change at every time The data is related to robotics system Now I need to write a python code which does the structuring of XML and eventually I will need to integrate with data base code How do I do?

I'll frame this as a reusable Python design: first build XML from sample/template fields, then later plug in database rows without changing XML logic.

Thought for a couple of seconds

You can do it in **3 layers**:

1. **Database layer** → fetch robotics data
2. **Mapping layer** → map database column names to OPC UA XML field names
3. **XML generator layer** → create XML using the same structure as sample XML

Recommended approach

Do **not hardcode values** inside XML. Keep the XML field names fixed, and pass values dynamically.

Example:

```
import xml.etree.ElementTree as ET
from datetime import datetime
```

```
def create_robotics_opcua_xml(data: dict) -> str:
    root = ET.Element("RoboticsData")
```

```
    robot = ET.SubElement(root, "Robot")
```

```
    ET.SubElement(robot, "RobotName").text = str(data.get("robot_name", ""))
```

```
    ET.SubElement(robot, "RobotID").text = str(data.get("robot_id", ""))
```

```
    ET.SubElement(robot, "OperationMode").text = str(data.get("operation_mode", ""))
```

```
    ET.SubElement(robot, "Status").text = str(data.get("status", ""))
```

```
    position = ET.SubElement(robot, "TCPPosition")
```

```
    ET.SubElement(position, "X").text = str(data.get("tcp_x", 0.0))
```

```
    ET.SubElement(position, "Y").text = str(data.get("tcp_y", 0.0))
```

```
    ET.SubElement(position, "Z").text = str(data.get("tcp_z", 0.0))
```

```
    orientation = ET.SubElement(robot, "TCPOrientation")
```

```
    ET.SubElement(orientation, "Rx").text = str(data.get("tcp_rx", 0.0))
```

```

ET.SubElement(orientation, "Ry").text = str(data.get("tcp_ry", 0.0))
ET.SubElement(orientation, "Rz").text = str(data.get("tcp_rz", 0.0))

joints = ET.SubElement(robot, "JointAngles")
for i in range(1, 7):
    ET.SubElement(joints, f"J{i}").text = str(data.get(f"j{i}", 0.0))

ET.SubElement(robot, "Timestamp").text = data.get(
    "timestamp",
    datetime.now().isoformat()
)

return ET.tostring(root, encoding="unicode")

```

Example database data

Later, your database code can return something like this:

```

db_data = {
    "robot_name": "Fanuc_R2000",
    "robot_id": "RB_001",
    "operation_mode": "AUTO",
    "status": "RUNNING",

    "tcp_x": 1200.5,
    "tcp_y": 340.2,
    "tcp_z": 890.0,

    "tcp_rx": 0.0,
    "tcp_ry": 1.57,
    "tcp_rz": 0.0,

    "j1": 10.5,
    "j2": -20.0,
    "j3": 35.2,
    "j4": 0.0,
    "j5": 90.0,
    "j6": 15.0
}

xml_output = create_robotics_opcua_xml(db_data)

print(xml_output)

```

Better structure for your real case

Since you already have a **sample OPC UA XML**, the best method is:

Sample XML field name	Database column name
RobotName	robot_name
RobotID	robot_id
OperationMode	mode
TCPPosition/X	tcp_x
TCPPosition/Y	tcp_y
TCPPosition/Z	tcp_z
JointAngles/J1	joint_1

Create a mapping dictionary:

```
FIELD_MAPPING = {  
    "RobotName": "robot_name",  
    "RobotID": "robot_id",  
    "OperationMode": "mode",  
    "Status": "robot_status",  
}
```

Then XML field names remain same as the sample, but values come from DB columns.

If you have sample XML

A very practical flow is:

- Read sample XML
- Find required tags
- Replace only the values
- Keep structure same
- Save new XML

Example:

```
import xml.etree.ElementTree as ET
```

```
def update_xml_template(template_path, output_path, data):  
    tree = ET.parse(template_path)  
    root = tree.getroot()  
  
    tag_to_db_key = {
```

```

"RobotName": "robot_name",
"RobotID": "robot_id",
"OperationMode": "operation_mode",
"Status": "status",
"X": "tcp_x",
"Y": "tcp_y",
"Z": "tcp_z",
"Rx": "tcp_rx",
"Ry": "tcp_ry",
"Rz": "tcp_rz",
"J1": "j1",
"J2": "j2",
"J3": "j3",
"J4": "j4",
"J5": "j5",
"J6": "j6",
}

```

```

for element in root.iter():
    if element.tag in tag_to_db_key:
        db_key = tag_to_db_key[element.tag]
        element.text = str(data.get(db_key, ""))

```

```

tree.write(output_path, encoding="utf-8", xml_declaration=True)

```

Usage:

```

update_xml_template(
    template_path="sample_opcua.xml",
    output_path="robotics_output.xml",
    data=db_data
)

```

My suggestion

Since you already have a sample XML, use the **template update approach**.

That means:

sample_opcua.xml + database row → generated_opcua.xml

This is safer because your XML structure will remain exactly like the OPC UA sample.

nodeset_demo_CurrentOne.xml

File

AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml

File

i can give you what is the current xml structure - the one which data base code has --> currently this is the way data base code is structring xml --> which is wrong also i am giving to be sample one --> this is sample template in which we want the data base code to be structure the xml

opcua-code.zip

Zip Archive

this is the current full data base code



Looks like currently they use this interface. Are they ?

AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml

File

are you sure test script outout xml will be exactly same as sample?

To preserve the XML **exactly byte-for-byte**, the only safe method is:

Read template file as plain text

Replace only value text between tags

Do not parse XML

Do not pretty-print

Do not write using XML libraries

So yes, the regex/text-replacement approach is the right one — but we must say this clearly:

Output XML = exact same template text
except the specific value characters changed

That means:

- same indentation
- same line breaks
- same attribute order
- same namespace prefixes
- same XML declaration
- same tags
- same structure

Use this approach, not `ElementTree` and not `lxml`.

The function should return the original template text with only selected value text replaced.

```
from pathlib import Path
from html import escape
import re
```

```
def generate_nodeset_xml_from_nodes(nodes, template_path: str) -> bytes:
    template_text = Path(template_path).read_text(encoding="utf-8")
    runtime_values = _extract_runtime_values_from_nodes(nodes)

    updated_text = update_template_values_exact(
        template_text=template_text,
        runtime_values=runtime_values,
    )

    return updated_text.encode("utf-8")
```

```
def update_template_values_exact(template_text: str, runtime_values: dict) -> str:
    result = template_text

    for field_name, value in runtime_values.items():
        result = replace_value_for_browsename(
            xml_text=result,
            browse_name=field_name,
            new_value=value,
        )
```

```
return result
```

```
def replace_value_for_browsename(xml_text: str, browse_name: str, new_value) -> str:
```

```
    escaped_value = escape(str(new_value))
```

```
    variable_pattern = re.compile(
```

```
rf'(<UAVariable\b[^>]*BrowseName="(?:\d+:)?{re.escape(browse_name)}"[\^>]*>.*?</UAVariable>)',
```

```
        re.DOTALL,
```

```
    )
```

```
    match = variable_pattern.search(xml_text)
```

```
    if not match:
```

```
        return xml_text
```

```
    variable_block = match.group(1)
```

```
    updated_block = replace_first_leaf_value(variable_block, escaped_value)
```

```
    return xml_text[:match.start(1)] + updated_block + xml_text[match.end(1):]
```

```
def replace_first_leaf_value(variable_block: str, new_value: str) -> str:
```

```
    value_match = re.search(
```

```
        r"(<Value>\s*.*?<uax:[A-Za-z0-9_]+\b[^>]*>)(.*?)(</uax:[A-Za-z0-9_]+>.*?\s*</Value>)",
```

```
        variable_block,
```

```
        re.DOTALL,
```

```
    )
```

```
    if not value_match:
```

```
        return variable_block
```

```
    return (
```

```
        variable_block[:value_match.start(2)]
```

```
        + new_value
```

```
        + variable_block[value_match.end(2):]
```

```
    )
```

```
def _extract_runtime_values_from_nodes(nodes) -> dict:
```

```

values = {}

for node in nodes:
    field_name = getattr(node, "label", None)

    if not field_name:
        field_name = getattr(node, "node_id", "")

    field_name = str(field_name).split(".")[-1].strip()

    if not field_name:
        continue

    try:
        latest_reading = node.readings.first()

        if latest_reading is None:
            continue

        if hasattr(latest_reading, "typed_value"):
            values[field_name] = latest_reading.typed_value
        else:
            values[field_name] = latest_reading.value

    except Exception:
        continue

return values

```

This gives:

Exact input template text

+

changed values only

But one condition:

The target field must already have a `<Value>...</Value>` block in the sample XML.

If a field has no `<Value>` block in the template, the code should not add one, because adding a new tag means the output is no longer exactly the same template.

my requirement is : Mock Django nodes + Sample XML template ↓ Value replacement only ↓
New XML generated

Yes — your requirement is exactly:

Mock/current Django nodes
+
Sample XML template
↓
Replace only existing values
↓
New XML generated

Use **plain text replacement**, not XML parsing.

Final behavior:

Template structure: exactly same
Formatting: exactly same
Only value text: changed

Example:

`<uax:Double>0</uax:Double>`

becomes:

`<uax:Double>42.5</uax:Double>`

Nothing else changes.

So the correct implementation is:

```
xml_bytes = generate_nodeset_xml_from_nodes(  
    nodes=mock_nodes,  
    template_path="AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml"  
)
```

where `nodes` can be:

`server.nodes.all()`

or mock nodes for testing.

Key condition:

The field must already exist in the sample template with a `<Value>` block.

If the sample template does not contain `<Value>` for a field, we should not create a new one, because that would change the template structure.

can you give the integratable code and test code?
Thought for a couple of seconds

Yes. Use this version. It keeps the **same current interface**:

```
nodes = server.nodes.all()
xml_data = generate_nodeset_xml_from_nodes(nodes, template_path)
```

It does **value replacement only**.

1. Create file:

opcuainterface/nodeset_template_formatter.py

```
from pathlib import Path
from html import escape
import re
```

```
def generate_nodeset_xml_from_nodes(nodes, template_path: str) -> bytes:
```

```
    """
```

```
    Current interface compatible:
```

```
        nodes = server.nodes.all()
```

```
    Behavior:
```

- Reads sample XML template as plain text
- Does NOT parse/rebuild XML
- Replaces only existing value text inside <Value>...</Value>
- Keeps template formatting/structure unchanged

```
    """
```

```
    template_text = Path(template_path).read_text(encoding="utf-8")
```

```
    runtime_values = _extract_runtime_values_from_nodes(nodes)
```

```
    updated_text = update_template_values_exact(
        template_text=template_text,
        runtime_values=runtime_values,
    )
```

```
    return updated_text.encode("utf-8")
```

```
def update_template_values_exact(template_text: str, runtime_values: dict) -> str:
    updated_text = template_text
```

```
    for field_name, value in runtime_values.items():
        updated_text = _replace_value_for_field(
            xml_text=updated_text,
            field_name=field_name,
            new_value=value,
        )
```

```
    return updated_text
```

```
def _replace_value_for_field(xml_text: str, field_name: str, new_value) -> str:
```

```
    """
```

```
    Finds:
```

```
        <UAVariable ... BrowseName="4:Temperature" ...>
```

```
        ...
```

```
        <Value>
```

```
            <uax:Double>0</uax:Double>
```

```
        </Value>
```

```
    </UAVariable>
```

```
    Replaces only:
```

```
        0 -> new_value
```

```
    """
```

```
    variable_pattern = re.compile(
```

```
rf'(<UAVariable\b[^\>]*BrowseName="(?:\d+)?{re.escape(field_name)}"[^\>]*>.*?</UAVariable>)',
    re.DOTALL,
)
```

```
    match = variable_pattern.search(xml_text)
```

```
    if not match:
```

```
        return xml_text
```

```
    variable_block = match.group(1)
```

```
    updated_block = _replace_first_value_text_only(
```

```
        variable_block=variable_block,
```

```
        new_value=_format_xml_value(new_value),
```

```
    )
```

```

return (
    xml_text[:match.start(1)]
    + updated_block
    + xml_text[match.end(1):]
)

```

```

def _replace_first_value_text_only(variable_block: str, new_value: str) -> str:
    """

```

Replaces only text inside the first uax value tag inside <Value>...</Value>.

Example:

```

<Value>
    <uax:Double>0</uax:Double>
</Value>

```

becomes:

```

<Value>
    <uax:Double>42.5</uax:Double>
</Value>

```

```

value_pattern = re.compile(
    r'(<Value>\s*. *?<uax:[A-Za-z0-9_]+\b[^\>]*>)(.*?)(</uax:[A-Za-z0-9_]+\>. *?</Value>)',
    re.DOTALL,
)

```

```

match = value_pattern.search(variable_block)

```

if not match:

```

    return variable_block

```

```

return (
    variable_block[:match.start(2)]
    + new_value
    + variable_block[match.end(2):]
)

```

```

def _extract_runtime_values_from_nodes(nodes) -> dict:
    """

```

Extracts values from current Django ORM node objects.

Expected:

node.label

node.readings.first().value

"""

values = {}

for node in nodes:

field_name = getattr(node, "label", None)

if not field_name:

field_name = getattr(node, "node_id", "")

field_name = str(field_name).split(".")[1].strip()

if not field_name:

continue

try:

latest_reading = node.readings.first()

if latest_reading is None:

continue

if hasattr(latest_reading, "typed_value"):

value = latest_reading.typed_value

else:

value = latest_reading.value

values[field_name] = value

except Exception:

continue

return values

def _format_xml_value(value) -> str:

if value is None:

return ""

if isinstance(value, bool):

return str(value).lower()

```
return escape(str(value))
```

2. Integrate in **views.py**

Use this inside your existing `export_nodes_xml()`:

```
from pathlib import Path
from django.conf import settings
from django.http import HttpResponse
from django.shortcuts import get_object_or_404
from django.views.decorators.http import require_http_methods

from .models import OPCUAServer
from .nodeset_template_formatter import generate_nodeset_xml_from_nodes

@require_http_methods(["GET"])
def export_nodes_xml(request, server_id: int):
    server = get_object_or_404(OPCUAServer, pk=server_id)

    nodes = server.nodes.all()

    template_path = (
        Path(settings.BASE_DIR)
        / "AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml"
    )

    xml_data = generate_nodeset_xml_from_nodes(
        nodes=nodes,
        template_path=str(template_path),
    )

    response = HttpResponse(xml_data, content_type="application/xml")
    response["Content-Disposition"] = (
        f'attachment; filename="nodeset_{server.name.replace(" ", "_")}.xml"'
    )

    return response
```

3. Test without database

Create this file at project root:

```
test_nodeset_template_formatter.py
```

```
from pathlib import Path
```

```
from opcuainterface.nodeset_template_formatter import (
    generate_nodeset_xml_from_nodes,
)
```

```
class MockReading:
```

```
    def __init__(self, value):
```

```
        self.value = value
```

```
        self.typed_value = value
```

```
class MockReadings:
```

```
    def __init__(self, value):
```

```
        self._value = value
```

```
    def first(self):
```

```
        return MockReading(self._value)
```

```
class MockNode:
```

```
    def __init__(self, label, value):
```

```
        self.label = label
```

```
        self.readings = MockReadings(value)
```

```
def main():
```

```
    template_path = "AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml"
```

```
    mock_nodes = [
```

```
        MockNode("Temperature", 42.5),
```

```
        MockNode("WorkOrder", "WO_123"),
```

```
        MockNode("TaskIdentifier", "TASK_001"),
```

```
        MockNode("MaxTorque", 150.0),
```

```
        MockNode("ActStatus", "Running"),
```

```
    ]
```

```
    xml_bytes = generate_nodeset_xml_from_nodes(
```

```
        nodes=mock_nodes,
```

```
        template_path=template_path,
```

)

```
output_path = Path("Generated_Test_NodeSet.xml")
output_path.write_bytes(xml_bytes)
```

```
print("Generated XML:", output_path.resolve())
```

```
if __name__ == "__main__":
    main()
```

Run:

```
python test_nodeset_template_formatter.py
```

This will create:

Generated_Test_NodeSet.xml

The output will keep the sample XML template the same and replace only matching existing `<Value>` contents.

AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml

File

are you sure test script output xml will be exactly same as sample?

Yes, you can use `lxml` with the sample template.

But the difference is:

lxml / xml tree:

Preserves XML structure correctly, but may not preserve byte-for-byte formatting.

Plain text replacement:

Preserves exact template text and changes only values.

For your requirement:

Sample XML template exactly same + only values change

the best option is still **plain text replacement**.

Use `lxml` only if you are okay with:

same XML structure

same OPC UA meaning

but possible minor formatting changes

Example `lxml` approach:

from `lxml` import `etree`

```
def generate_nodeset_xml_from_nodes_lxml(nodes, template_path):
```

```
    parser = etree.XMLParser(remove_blank_text=False)
```

```
    tree = etree.parse(template_path, parser)
```

```
    ns = {
```

```
        "ua": "http://opcfoundation.org/UA/2011/03/UANodeSet.xsd"
```

```
    }
```

```
    runtime_values = _extract_runtime_values_from_nodes(nodes)
```

```
    for variable in tree.xpath("//ua:UAVariable", namespaces=ns):
```

```
        browse_name = variable.get("BrowseName", "")
```

```
        field_name = browse_name.split(":")[-1]
```

```

if field_name not in runtime_values:
    continue

value = variable.xpath("./ua:Value", namespaces=ns)

if not value:
    continue

value_element = value[0]

for child in value_element.iter():
    if child is not value_element and len(child) == 0:
        child.text = str(runtime_values[field_name])
        break

return etree.tostring(
    tree,
    encoding="utf-8",
    xml_declaration=True,
    pretty_print=False
)

```

But again, for **exact same template text**, use your plain-text replacement version.

My recommendation:

Production OPC UA correctness → lxml is okay

Exact file/template preservation → plain text replacement is better

For your specific requirement, I would keep the text-based version.

can you give the code which uses 1. lxml 2. xmltree in this case xml structure and field names will be always sp t then what is problem in using lxml or xmltree?

Thought for a couple of seconds

Yes, you can use both. Since your XML **structure and field names are fixed**, `lxml` or `xml.etree.ElementTree` is technically fine.

The only issue is:

lxml / ElementTree may slightly change formatting

But they will preserve the **XML structure and field names correctly**.

1. `lxml` version

Install:

```
pip install lxml
```

```
from pathlib import Path
```

```
from lxml import etree
```

```
UA_NS = "http://opcfoundation.org/UA/2011/03/UANodeSet.xsd"
```

```
NS = {  
    "ua": UA_NS,  
}
```

```
def generate_nodeseet_xml_from_nodes_lxml(nodes, template_path: str) -> bytes:
```

```
    parser = etree.XMLParser(  
        remove_blank_text=False,  
        resolve_entities=False  
    )
```

```
    tree = etree.parse(template_path, parser)
```

```
    root = tree.getroot()
```

```
    runtime_values = _extract_runtime_values_from_nodes(nodes)
```

```
    for variable in root.xpath("//ua:UAVariable", namespaces=NS):
```

```
        browse_name = variable.get("BrowseName", "")
```

```
        field_name = browse_name.split(":")[-1]
```

```
        if field_name not in runtime_values:
```

```
            continue
```

```
        value_elements = variable.xpath("./ua:Value", namespaces=NS)
```

```
        if not value_elements:
```

```
            continue
```

```
        value_element = value_elements[0]
```

```
leaf = _find_first_leaf(value_element)
```

```
if leaf is not None:
```

```
    leaf.text = _format_value(runtime_values[field_name])
```

```
return etree.tostring(
```

```
    tree,
```

```
    encoding="utf-8",
```

```
    xml_declaration=True,
```

```
    pretty_print=False
```

```
)
```

```
def _find_first_leaf(element):
```

```
    for child in element.iter():
```

```
        if child is element:
```

```
            continue
```

```
    if len(child) == 0:
```

```
        return child
```

```
    return None
```

```
def _extract_runtime_values_from_nodes(nodes) -> dict:
```

```
    values = {}
```

```
    for node in nodes:
```

```
        field_name = getattr(node, "label", None)
```

```
        if not field_name:
```

```
            field_name = getattr(node, "node_id", "")
```

```
        field_name = str(field_name).split(".")[1].strip()
```

```
        if not field_name:
```

```
            continue
```

```
        try:
```

```
            latest_reading = node.readings.first()
```

```
            if latest_reading is None:
```

```
                continue
```

```
            if hasattr(latest_reading, "typed_value"):
```

```
                value = latest_reading.typed_value
```

```
            else:
```

```
                value = latest_reading.value
```

```

        values[field_name] = value

    except Exception:

        continue

    return values

def _format_value(value) -> str:

    if value is None:

        return ""

    if isinstance(value, bool):

        return str(value).lower()

    return str(value)

```

2. **xml.etree.ElementTree** version

No extra installation required.

```
import xml.etree.ElementTree as ET
```

```
UA_NS = "http://opcfoundation.org/UA/2011/03/UANodeSet.xsd"
```

```
NS = {  
    "ua": UA_NS,  
}
```

```
ET.register_namespace("", UA_NS)
```

```
ET.register_namespace("uax", "http://opcfoundation.org/UA/2008/02/Types.xsd")
```

```
def generate_nodeset_xml_from_nodes_xmltree(nodes, template_path: str) -> bytes:
```

```
    tree = ET.parse(template_path)
```

```
    root = tree.getroot()
```

```
    runtime_values = _extract_runtime_values_from_nodes(nodes)
```

```
    for variable in root.findall("./ua:UAVariable", NS):
```

```
        browse_name = variable.attrib.get("BrowseName", "")
```

```
        field_name = browse_name.split(":")[-1]
```

```
        if field_name not in runtime_values:
```

```
            continue
```

```
        value_element = variable.find("ua:Value", NS)
```



```
if value_element is None:
    continue

leaf = _find_first_leaf(value_element)

if leaf is not None:
    leaf.text = _format_value(runtime_values[field_name])

return ET.tostring(
    root,
    encoding="utf-8",
    xml_declaration=True
)
```

```
def _find_first_leaf(element):
    for child in element.iter():
        if child is element:
            continue

        if len(list(child)) == 0:
            return child

    return None
```

```
def _extract_runtime_values_from_nodes(nodes) -> dict:
```

```
    values = {}
```

```
    for node in nodes:
```

```
        field_name = getattr(node, "label", None)
```

```
        if not field_name:
```

```
            field_name = getattr(node, "node_id", "")
```

```
        field_name = str(field_name).split(".")[1].strip()
```

```
        if not field_name:
```

```
            continue
```

```
        try:
```

```
            latest_reading = node.readings.first()
```

```
            if latest_reading is None:
```

```
                continue
```

```
            if hasattr(latest_reading, "typed_value"):
```

```
                value = latest_reading.typed_value
```

```
    else:

        value = latest_reading.value

    values[field_name] = value

except Exception:

    continue

return values


def _format_value(value) -> str:

    if value is None:

        return ""

    if isinstance(value, bool):

        return str(value).lower()

    return str(value)
```

What is the problem in using **lxml** or **xmltree**?

No problem if your requirement is:

same XML structure

same field names

same OPC UA meaning

values updated

But there is a problem only if your requirement is:

output file must look byte-by-byte exactly like input template

Because XML libraries may change:

spacing

indentation

namespace prefix ordering

self-closing tags

XML declaration style

attribute ordering

So my recommendation:

For robust XML handling: use `lxml`

For exact text preservation: use plain text replacement

For your case, if exact visual formatting is not checked by another tool, use **`lxml`**. It is cleaner and safer than regex.

in both cases can you give test code and integratable code?

Thought for a couple of seconds

Yes. Below are both options: **`lxml`** and **`xml.etree.ElementTree`**. Both keep the current interface:

```
nodes = server.nodes.all()
```

```
xml_data = generate_nodeset_xml_from_nodes(nodes, template_path)
```

Option 1: **`lxml`** integratable code

Create:

```
opcuainterface/nodeset_template_formatter_lxml.py
```

```
from lxml import etree
```

```
UA_NS = "http://opcfoundation.org/UA/2011/03/UANodeSet.xsd"
```

```
NS = {
```

```
    "ua": UA_NS,
```

```
}
```

```
def generate_nodeset_xml_from_nodes(nodes, template_path: str) -> bytes:
```

```
    parser = etree.XMLParser(
```

```
        remove_blank_text=False,
```

```
        resolve_entities=False,
```

```
)
```

```
tree = etree.parse(template_path, parser)
```

```
root = tree.getroot()
```

```
runtime_values = _extract_runtime_values_from_nodes(nodes)
```

```
for variable in root.xpath("./ua:UAVariable", namespaces=NS):
```

```
    browse_name = variable.get("BrowseName", "")
```

```

field_name = browse_name.split(":-")[-1]

if field_name not in runtime_values:
    continue

value_elements = variable.xpath("./ua:Value", namespaces=NS)

if not value_elements:
    continue

leaf = _find_first_leaf(value_elements[0])

if leaf is not None:
    leaf.text = _format_value(runtime_values[field_name])

return etree.tostring(
    tree,
    encoding="utf-8",
    xml_declaration=True,
    pretty_print=False,
)

def _find_first_leaf(element):

```

```
for child in element.iter():
```

```
    if child is element:
```

```
        continue
```

```
    if len(child) == 0:
```

```
        return child
```

```
return None
```

```
def _extract_runtime_values_from_nodes(nodes) -> dict:
```

```
    values = {}
```

```
    for node in nodes:
```

```
        field_name = getattr(node, "label", None)
```

```
        if not field_name:
```

```
            field_name = getattr(node, "node_id", "")
```

```
        field_name = str(field_name).split(".")[1].strip()
```

```
        if not field_name:
```

```
            continue
```

```
latest_reading = node.readings.first()
```

```
if latest_reading is None:
```

```
    continue
```

```
if hasattr(latest_reading, "typed_value"):
```

```
    values[field_name] = latest_reading.typed_value
```

```
else:
```

```
    values[field_name] = latest_reading.value
```

```
return values
```

```
def _format_value(value) -> str:
```

```
    if value is None:
```

```
        return ""
```

```
    if isinstance(value, bool):
```

```
        return str(value).lower()
```

```
    return str(value)
```

Install:

```
pip install lxml
```

Option 2: `xml.etree.ElementTree` integratable code

Create:

`opcuainterface/nodeset_template_formatter_xmltree.py`

```
import xml.etree.ElementTree as ET
```

```
UA_NS = "http://opcfoundation.org/UA/2011/03/UANodeSet.xsd"
```

```
UAX_NS = "http://opcfoundation.org/UA/2008/02/Types.xsd"
```

```
NS = {  
    "ua": UA_NS,  
}
```

```
ET.register_namespace("", UA_NS)
```

```
ET.register_namespace("uax", UAX_NS)
```

```
def generate_nodeset_xml_from_nodes(nodes, template_path: str) -> bytes:
```

```
    tree = ET.parse(template_path)
```

```
    root = tree.getroot()
```

```
    runtime_values = _extract_runtime_values_from_nodes(nodes)
```

```
    for variable in root.findall("./ua:UAVariable", NS):
```

```

browse_name = variable.attrib.get("BrowseName", "")
field_name = browse_name.split(":")[-1]

if field_name not in runtime_values:
    continue

value_element = variable.find("ua:Value", NS)

if value_element is None:
    continue

leaf = _find_first_leaf(value_element)

if leaf is not None:
    leaf.text = _format_value(runtime_values[field_name])

return ET.tostring(
    root,
    encoding="utf-8",
    xml_declaration=True,
)

def _find_first_leaf(element):

```

```
for child in element.iter():
```

```
    if child is element:
```

```
        continue
```

```
    if len(list(child)) == 0:
```

```
        return child
```

```
return None
```

```
def _extract_runtime_values_from_nodes(nodes) -> dict:
```

```
    values = {}
```

```
    for node in nodes:
```

```
        field_name = getattr(node, "label", None)
```

```
        if not field_name:
```

```
            field_name = getattr(node, "node_id", "")
```

```
        field_name = str(field_name).split(".")[1].strip()
```

```
        if not field_name:
```

```
            continue
```

```
latest_reading = node.readings.first()
```

```
if latest_reading is None:
```

```
    continue
```

```
if hasattr(latest_reading, "typed_value"):
```

```
    values[field_name] = latest_reading.typed_value
```

```
else:
```

```
    values[field_name] = latest_reading.value
```

```
return values
```

```
def _format_value(value) -> str:
```

```
    if value is None:
```

```
        return ""
```

```
    if isinstance(value, bool):
```

```
        return str(value).lower()
```

```
    return str(value)
```

Integrate in **views.py**

Use either one.

For `lxml`:

```
from .nodeset_template_formatter_lxml import generate_nodeset_xml_from_nodes
```

For `xmltree`:

```
from .nodeset_template_formatter_xmltree import generate_nodeset_xml_from_nodes
```

Then your view:

```
from pathlib import Path
```

```
from django.conf import settings
```

```
from django.http import HttpResponse
```

```
from django.shortcuts import get_object_or_404
```

```
from django.views.decorators.http import require_http_methods
```

```
from .models import OPCUAServer
```

```
from .nodeset_template_formatter_lxml import generate_nodeset_xml_from_nodes
```

```
# or:
```

```
# from .nodeset_template_formatter_xmltree import generate_nodeset_xml_from_nodes
```

```
@require_http_methods(["GET"])
```

```
def export_nodes_xml(request, server_id: int):
```

```
    server = get_object_or_404(OPCUAServer, pk=server_id)
```

```
    nodes = server.nodes.all()
```

```
    template_path = (
```

```

    Path(settings.BASE_DIR)

    / "AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml"

)

xml_data = generate_nodeset_xml_from_nodes(

    nodes=nodes,

    template_path=str(template_path),

)

response = HttpResponse(xml_data, content_type="application/xml")

response["Content-Disposition"] = (

    f'attachment; filename="nodeset_{server.name.replace(" ", "_")}.xml"'

)

return response

```

Test code for both

Create:

test_nodeset_template_formatter.py

from pathlib import Path

Choose one:

```

from opcuainterface.nodeset_template_formatter_lxml import
generate_nodeset_xml_from_nodes

```

```
# from opcuainterface.nodeset_template_formatter_xmltree import  
generate_nodeset_xml_from_nodes
```

```
class MockReading:
```

```
    def __init__(self, value):
```

```
        self.value = value
```

```
        self.typed_value = value
```

```
class MockReadings:
```

```
    def __init__(self, value):
```

```
        self._value = value
```

```
    def first(self):
```

```
        return MockReading(self._value)
```

```
class MockNode:
```

```
    def __init__(self, label, value):
```

```
        self.label = label
```

```
        self.readings = MockReadings(value)
```

```
def main():
```

```
template_path = "AtlasCopco-Tools.Nodeset2_ToBeWantedSample.xml"
```

```
mock_nodes = [  
    MockNode("Temperature", 42.5),  
    MockNode("WorkOrder", "WO_123"),  
    MockNode("TaskIdentifier", "TASK_001"),  
    MockNode("MaxTorque", 150.0),  
    MockNode("ActStatus", "Running"),  
]
```

```
xml_bytes = generate_nodeset_xml_from_nodes(  
    nodes=mock_nodes,  
    template_path=template_path,  
)
```

```
output_path = Path("Generated_Test_NodeSet.xml")  
output_path.write_bytes(xml_bytes)
```

```
print("Generated XML:", output_path.resolve())
```

```
if __name__ == "__main__":  
    main()
```

Run:


```
python test_nodeseet_template_formatter.py
```

Recommendation: use **lxml** for your case. It handles complex OPC UA XML better than `xml.etree`.